

Consolidated services server — minimum specification

The server itself is already on hand (see docs/topology.md) — this document calculates the **target minimum spec** to check that hardware against, working forward from the actual workload this box runs (docs/topology.md 's VM/container table) rather than a generic "decent server" guess. Where a figure depends on something this repo hasn't pinned down (the event's actual day count/hours), that's flagged explicitly rather than assumed — see docs/open-questions.md for the running list.

Storage — three pools, not one array

Unraid's traditional parity array is built around mixed-size spinning disks with a single-write-at-a-time parity bottleneck, absorbed by an SSD cache pool that the overnight "mover" drains onto the array — a good fit for bulk archival storage, a poor fit for a box whose entire storage workload here is already flash. Recommendation: skip the legacy array entirely and use three separate Unraid **pools** (ZFS, available since Unraid 6.12), each sized and mirrored for what it actually holds.

Container/VM pool

Holds docker.img , all container appdata (including the three databases — MariaDB, MongoDB, Redis — that want fast, low-latency small-I/O access), and the BirdDog Central VM's vdisk.

Item	Estimate
BirdDog Central vdisk	~80 GB (OS + app)
docker.img	~50 GB
Combined appdata (12 containers + the Tailscale subnet router + 3 DBs)	~58 GB
Total	~188 GB

Minimum: 500 GB usable, mirrored NVMe (2× ~512GB+ NVMe) — comfortable headroom over the ~188 GB estimate, and mirrored because rebuilding a Windows VM from scratch mid-event is real downtime, not just an inconvenience. NVMe specifically (not SATA SSD) because VM and database performance is latency-sensitive, not throughput-bound.

Drive class: mainstream consumer NVMe with onboard DRAM cache (not a DRAM-less budget drive) is genuinely sufficient here — no need for the enterprise tier the recording pool below needs. VM/appdata churn is nowhere near enough sustained write volume to meaningfully test even consumer-grade endurance ratings, and the workload cares about consistent low-queue-depth latency (how snappy a container restart or a DB query feels), which good consumer NVMe already delivers.

Content pool — 500 GB (as specified)

Nextcloud's own storage: theatre laptop content synced via rclone (established at 10-20 GB/theatre × 12 theatres = **120-240 GB**, see docs/bandwidth-analysis.md), plus Nextcloud's own app/database footprint and versioning overhead (flagged as a real multiplier in docs/atem-iso-ingest.md 's versioning caveat — version retention settings directly affect how much of this budget versioning eats into). 500 GB gives roughly 2-4x headroom over the raw 120-240 GB content estimate, which comfortably absorbs version history without needing an aggressive retention policy from day one.

Recommendation: mirrored, SATA SSD is adequate (no NVMe-level latency requirement here) — mirrored less for data-loss prevention (every laptop is still the source of truth for its own content even if this copy is lost) and more for **availability**: a lost content pool mid-event means re-running the initial bulk sync for all 12 theatres, which is itself only a ~20-40 minute operation at 50-100 Mbps (docs/bandwidth-analysis.md) but is a real, avoidable disruption during a live event.

Drive class: same reasoning as the container/VM pool — mainstream consumer SATA SSD with DRAM cache, the cheapest reasonable tier that still isn't a DRAM-less budget part. Gentle, sporadic write pattern; no case for spending more here.

Recording pool — 8 TB, all-flash (as specified)

This is where the calculation actually matters — both *why* it has to be flash and *whether 8 TB is enough*.

Why all-flash, not spinning disk — it's not about raw throughput. Combined worst-case write load onto this pool, from docs/bandwidth-analysis.md 's own established figures:

Source	Per-unit worst case	Count	Total
ATEM ISO ingest	~90.4 Mbps (94 Mbps incl. WireGuard tax)	12 theatres	~1,085 Mbps
VMix Record ingest	~30 Mbps (assumed, unconfirmed — see docs/open-questions.md)	4 PCs	~120 Mbps
Combined			~1,205 Mbps ≈ 151 MB/s

151 MB/s sustained is well within a single 7200 RPM HDD's *rated sequential* throughput on paper — so the case for flash isn't the aggregate number, it's the **access pattern**. That 151 MB/s isn't one stream, it's **16 independent files being appended to concurrently** (12 ATEM + 4 VMix, each on its own ~60-90s poll cycle — see docs/atem-iso-ingest.md and docs/vmix-record-ingest.md), with Nextcloud's own targeted `occ files:scan` indexing calls layered on top of that. A spinning disk pays a real seek penalty (~5-15ms) every time it switches between those 16+ scattered write targets — under this specific concurrent-scattered pattern, achievable HDD throughput collapses to a small fraction of its rated sequential number, while any flash drive (no seek penalty) handles 151 MB/s of concurrent random-ish writes without strain. The "all-flash" requirement is about the write *pattern* this design creates, not the volume.

Does 8 TB actually cover the event? This is the one figure genuinely blocked on an input this repo doesn't have: **actual event duration (days × active-recording hours/day) isn't established anywhere in this design**. Working from the same combined-load figures:

	Combined rate	8 TB covers
Realistic (5-7 active ATEM channels/theatre, per docs/bandwidth-analysis.md)	~99.4 MB/s ≈ 349 GB/hr	~23.5 hours
Worst case (all 9 ATEM channels/theatre)	~150.6 MB/s ≈ 530 GB/hr	~15.5 hours

At a typical ~8-10 hour active-program day, that's roughly **1.5-3 days** of continuous worst-to-realistic-case recording before the pool fills — workable for a short event, tight for a longer one, and this assumes every theatre is actually near its "realistic" active-channel count for the whole day, which per the original design intent ("likely only 4-6 [channels] will actually have any data") may be conservative. **Confirm actual event length against this table before treating 8 TB as settled** — if it's a multi-day event without a periodic archive-off step already planned, either the day count needs to fit the budget above, or a nightly archive-to-cold-storage step needs adding to free capacity for the next day. The ingest pipelines write directly into Nextcloud's External Storage location and, in the same pull, the edit-suite NAS (docs/atem-iso-ingest.md , docs/live-editing.md) — neither write has an offload stage that frees *this* pool. Tracked in docs/open-questions.md .

Redundancy — worth adding capacity for, not assumed in the 8 TB figure. This pool is the authoritative archive: the dual-write to the edit-suite NAS (docs/live-editing.md) does give the footage a second landing spot at ingest time, but that copy may be a curated subset (open-questions #12) and one of the candidate NAS units is RAID 0 — so a failure here still risks footage that can't be re-shot. Two ways to get 8 TB *usable* with 1-drive fault tolerance:

Option	Raw capacity needed	Trade-off
2× 8 TB NVMe, mirrored	16 TB raw	Simplest to reason about, fastest resilver, 50% capacity overhead
4× ~2.7 TB NVMe, RAID-Z1	~10.8 TB raw	75% capacity efficiency vs. 50%, slower resilver on failure, more drives to manage

Recommend the mirror for the same reason Unraid was chosen over TrueNAS SCALE elsewhere in this design (docs/topology.md) — operational simplicity, since this may be maintained by AV staff rather than a storage specialist, matters more here than squeezing out the last few TB of efficiency.

Endurance. Since this repo already treats the router hardware as reusable across events (see docs/gl-inet-rationale.md), the same applies here — a full pool fill is roughly 8 TB of actual flash writes (these are byte-range *appends*, not whole-file rewrites, so total written ≈ total recorded, not a multiple of it), and a mirrored pair each independently absorb that same 8 TB per event. Even a deliberately-generous worst-case estimate — ~530 GB/hr sustained for a full 24 h/day across ~20 event-days a year, beyond what the pool could even hold without nightly archive-off — comes to roughly 260 TB/year of actual writes to the pool — comfortably inside what even the *lower* enterprise endurance tier (see below) is rated for over a normal warranty period, so endurance headroom isn't actually the tight constraint here once the right drive class is picked.

Drive class: this is the one pool where consumer-grade flash — even a good, DRAM-cached, high-TBW consumer drive — isn't the right call, and endurance isn't the reason. The deciding factor is **power-loss protection (PLP)**: capacitor-backed circuitry that flushes a drive's write cache to permanent storage if power drops mid-write. Consumer/prosumer NVMe drives don't have this. A generator hiccup, a tripped breaker, or a UPS transfer glitch mid-event — exactly the kind of thing a live event's own power setup can produce — risks losing whatever was sitting in the drive's write cache at that instant, on the one pool in this whole design with no second copy of the data anywhere else. That's the actual argument for **enterprise-class NVMe** here specifically (not "enterprise is always better," which isn't true for the other two pools above) — server-validated drives with capacitor-backed PLP, provisioned for sustained high-queue-depth writes without the cache-cliff behavior consumer drives show once their fast write cache fills under prolonged concurrent load. Enterprise drives are also sold in more than one endurance tier

(commonly around 1 drive-write-per-day vs. 3x that for a heavier-duty tier) — the lighter of the two tiers already has large margin over this workload's realistic annual write volume, so there's no need to pay for the heavier tier's endurance on top of the PLP requirement that's actually driving this choice.

Write caching

With every pool already flash (no legacy spinning array behind anything), the classic Unraid "SSD cache pool absorbing writes before the overnight mover" pattern doesn't apply here — there's no slow array for it to shield. What actually buffers the bursty, 16-concurrent-stream ingest pattern described above is:

- **RAM** — the OS page cache smooths write bursts before they hit the drives (see the RAM section below, which already budgets for this).
- **The drives' own onboard DRAM cache** — a real purchasing consideration: choose NVMe drives with onboard DRAM (not DRAM-less/QLC-only budget drives), since sustained multi-stream concurrent writes are exactly the workload DRAM-less drives handle worst.

NIC — validating the already-decided 2x bonded GbE

`docs/topology.md` already settled on 2x gigabit NICs, bonded (802.3ad LACP). Checking that against the heaviest combined load this design has identified — the mass-NDI-fallback disaster scenario from `docs/bandwidth-analysis.md`, layered on top of full ATEM ingest:

Direction	Load	Total
Inbound (theatres → mothership)	ATEM ingest worst case (~1,123 Mbps) + VMix ingest (~120 Mbps) + ATEM Overseer (~125 Mbps) + Flock (~125 Mbps)	~1,493 Mbps
Outbound (mothership → theatres), all 12 theatres on NDI fallback simultaneously	12 x ~130 Mbps	~1,560 Mbps

(Inbound total updated from an earlier ~1,205 Mbps once ATEM Overseer's and Flock's monitoring/preview streams were added — see `docs/topology.md` — each contributing ~125 Mbps fleet-wide on top of what was already accounted for.)

Both directions run independently over a full-duplex link, so they don't stack against each other — but each direction needs to fit across the bond's two physical 1 Gbps links via LACP's per-flow hashing (each *individual* flow is capped at 1 Gbps, since LACP doesn't split one flow across both links). With 14+ independent theatre flows in each direction (12 ATEM ingest + 12 Overseer + 12 Flock inbound, each individually well under 1 Gbps — worst case ~94 Mbps ATEM, ~10.4 Mbps apiece for Overseer/Flock, ~130 Mbps NDI outbound), the hash has plenty of separate flows to distribute across both links without any single one bottlenecking — **the existing 2x 1GbE bonded design still checks out, including against the worst-case disaster scenario**, not just normal operation. Margin has narrowed since this was first validated (~1,205 → ~1,493 Mbps inbound, both still comfortably under the bond's ~2,000 Mbps aggregate), the same erosion `docs/bandwidth-analysis.md` tracks everywhere else Overseer and Flock touch this design — see that doc's own worked-through comparison of this exact disaster scenario with and without pausing ATEM ingest fleet-wide, which is the more consequential mitigation than the NIC choice itself.

One load deliberately NOT counted against this bond: the dual-write to the edit-suite NAS. The same ingest containers that produce the inbound figures above also write every recording out again to the edit-suite NAS at `192.168.22.2 ( docs/live-editing.md )` — up to another ~1,243 Mbps of outbound at worst case. If that leg transited the theatre-facing bond it would stack on top of the NDI-fallback outbound and break the conclusion above — so it must **not**: the edit suite is its own 10GbE LAN, and this box needs a separate edit-LAN-facing interface (a 10GbE NIC, or at minimum its own dedicated port) for that traffic, counted in the spec alongside the 2× bonded theatre-facing GbE.

Worth it anyway: 2× 2.5GbE, if the box/switch already support it at no real added cost. Doesn't change the conclusion above, but many current motherboards ship 2.5GbE onboard already, and it buys real margin against the one thing this analysis can't fully account for — LACP hash distribution isn't perfectly even in practice, and real headroom above a "checks out, but not by a huge margin" number is cheap insurance if it's already sitting on the board.

RAM

Workload	Estimate	Basis
BirdDog Central VM	8 GB	Routing/control app, not encode
Nextcloud + MariaDB + Redis	6 GB	InnoDB buffer pool + PHP workers + Redis cache
UniFi Controller + MongoDB	3 GB	Manages only the Cloud Gateway (the 14 GL-iNet routers are GLKVM-Cloud's, not UniFi's) — MongoDB's WiredTiger cache doesn't need much at this scale
ATEM Overseer + Flock	5 GB	Unlike the other admin/control-plane containers below, these two are actually decoding video — up to 12 concurrent theatre streams apiece for their monitoring dashboards (see <code>docs/bandwidth-analysis.md</code>), not just pushing config or metadata
Restreamer, NDI Discovery, DERP, both ingest containers, GLKVM-Cloud (rttys+coturn), ATEM Fleet Admin, Tailscale router	9 GB	9 lightweight containers, ~1 GB each budgeted
Unraid OS + ZFS ARC (3 pools, ~8.5 TB combined)	8 GB	Soft ZFS guidance is roughly 1 GB RAM per TB of pool for decent ARC hit rate — this is a floor, not a hard requirement, ZFS is adaptive
Subtotal	~39 GB	Down from ~71 GB before the VMix instance VM (32 GB on its own) was removed from the design
Minimum, with headroom	64 GB	39 GB doesn't map to a clean multi-channel ECC DIMM configuration — 64 GB is the next practical capacity above it with real margin, not just rounding up to the subtotal
Recommended	96 GB	Gives ZFS ARC meaningfully more room across all three pools, and covers any future container additions without revisiting the DIMM population

ECC, not just capacity. All three storage pools above are ZFS — ZFS leans on RAM for checksumming and its ARC read cache, and a bit flip in ordinary (non-ECC) RAM can get silently written into a checksum or into the recording pool's parity/mirror data before anyone notices, which defeats the whole point of using ZFS for the one pool holding unrepeatable footage. ECC RAM catches and corrects that class of error before it propagates. Pair this with a platform that has *validated* ECC support (see the CPU section below) — ECC only protects against silent corruption if it's actually enabled and working, which isn't guaranteed on every board that merely accepts ECC modules electrically.

CPU

Workload	Estimate	Basis
BirdDog Central	2 cores	Routing/control, not encode
Docker stack (12 containers + the Tailscale subnet router + 3 DBs)	5-7 cores shared	Mostly I/O-bound; Restreamer's SRT relay (remux, not transcode, per this design's primary path) and ATEM Overseer + Flock (each decoding up to 12 concurrent monitoring streams) are the heaviest individual containers
Unraid OS/array overhead	2 cores	
Subtotal	9-11 cores	Down from 17-19 before the VMix instance VM (8 dedicated cores on its own) was removed from the design
Minimum	12 cores / 24 threads, high sustained (not just boost) clock	Rounds above the subtotal's high end — see platform-class discussion below, which matters more than squeezing out the last core or two

The platform class matters more than the core count, and this is worth spelling out rather than just naming a chip. A 16-core desktop CPU is easy to find — the actual constraint is everything *around* it:

- **PCIe lanes.** With up to 6 NVMe drives spread across the three pools above, this box still wants 20-30+ usable PCIe lanes — though the pressure has eased since the VMix VM (and with it a mandatory ×16 GPU slot) left the design. Mainstream consumer desktop platforms (the socket a typical Ryzen 9 or Core i9/Ultra 9 sits in) are now workable with careful drive placement and bifurcation; a **workstation/HEDT-class platform** (Threadripper PRO or a high-end Xeon W) still clears it without any compromise, and remains the safer choice — but it's now a preference with a real mainstream alternative, no longer the only category that fits.
- **Validated ECC.** Some mainstream consumer boards technically accept ECC memory electrically, but whether ECC actually *functions* — gets detected, enabled, and does correction — varies board-to-board and isn't something to gamble on for the pool holding irreplaceable footage. Workstation platforms build ECC support into the platform itself, officially validated, not a maybe.
- **Sustained clock over peak boost.** The heavy loads here are continuous, not bursty — ATEM Overseer and Flock each decode up to 12 monitoring streams for the whole show, and the ingest pipelines run all day — so a CPU's *base* clock under sustained all-core load is a better predictor than its headline single-core boost number. Worth actually comparing base clocks across whichever specific chips are shortlisted, not just core count.

Trade-off worth being upfront about: workstation/HEDT platforms cost meaningfully more (CPU + motherboard together) than the mainstream desktop alternative, and idle power draw for 24/7 operation runs higher too. That's a real cost, not a rounding error. Since the VMix VM left the design, a high-end mainstream build with *validated* (not merely tolerated) ECC support is a legitimate cheaper alternative — the deciding check is the ECC validation and having enough lanes for the NVMe pools, not the platform label.

(IOMMU/VT-d support — previously a hard requirement here for GPU passthrough to the VMix VM — is no longer required by anything in the design. Virtually all current platforms have it anyway; it just no longer needs specific confirmation.)

GPU — no longer required

An earlier revision of this design carried a mothership VMix instance VM with a passthrough GPU, and this section specified a workstation-class NVENC card for it. **That VM has been removed** (docs/open-questions.md #10 — its role was never established; the theatre program feeds originate from the VMix node PCs, which have their own hardware). With it gone, nothing on this box needs a GPU:

- **Restreamer** relays SRT by remuxing, not transcoding — no encode.
- **BirdDog Central** is NDI routing/control — no encode.
- **ATEM Overseer and Flock** decode up to 12 monitoring/preview streams each, but these are 10 Mbps H.264 previews — CPU decode at this scale is already budgeted in the CPU table above.

The "decent" discrete GPU already in the box (docs/topology.md) can stay as spare capacity — potentially useful later for container-level decode/encode offload (e.g. if Overseer/Flock support hardware acceleration, or a future transcode workload appears) — but the spec no longer *requires* any GPU, and no replacement or upgrade should be budgeted for one.

Summary — minimum spec to check the existing server against

Component	Minimum	Why this class
CPU	12 cores / 24 threads, strong sustained clock — workstation/HEDT platform preferred, high-end mainstream with validated ECC acceptable	PCIe lane count for up to 6 NVMe drives, validated ECC support; lane pressure eased since the VMix VM (and its x16 GPU slot) left the design
Motherboard	Validated ECC support, enough PCIe 4.0/5.0 lanes for 3 NVMe pools	Same reasoning as CPU — the two are a package
RAM	64 GB minimum, 96 GB recommended, ECC, populate all available channels	ZFS data integrity across all 3 pools; subtotal ~39 GB since the VMix VM's 32 GB left the budget
GPU	None required — the on-hand GPU stays as spare capacity only	Nothing in the design encodes on this box any more; see the GPU section
Container/VM pool	500 GB usable, mirrored, consumer-grade DRAM-cached NVMe	Latency-sensitive VM/DB workload; consumer endurance is more than enough
Content pool	500 GB usable, mirrored, consumer-grade DRAM-cached SATA SSD	Gentle Nextcloud file I/O; no case for spending more
Recording pool	8 TB usable, mirrored (16 TB raw), enterprise-grade NVMe with power-loss protection	PLP is the deciding factor — this pool is the authoritative archive (the edit-suite NAS dual-write copy may be a subset, and may be RAID 0 — see docs/live-editing.md); confirm event duration against the capacity table above first
Network	2x 1GbE, bonded LACP (already decided, validated above) — 2x 2.5GbE if free to obtain — plus a separate edit-LAN-facing interface (10GbE) for the dual-write leg	Checks out even against the mass-NDI-fallback disaster scenario, provided the edit-suite dual-write never rides the theatre-facing bond

Everything above is a calculated **target**, not a purchase order — the actual box is already on hand per docs/topology.md . Next step is checking its real spec against this table and updating docs/open-questions.md #1 with the result.